Adaptive versus fixed surrogate gradient methods in SNNs

Antonio Jurjević,
Faculty of Science, Split, Croatia

ABSTRACT

Spiking neural networks (SNNs) offer an energy efficient, biologically inspired alternative to conventional neural networks. but training them requires surrogate gradient methods to approximate non-differentiable spike events. This work compares an adaptive surrogate gradient method (AdaLi) to a fixed sigmoid-based surrogate across two SNN architectures and three datasets (MNIST, CIFAR-10, CIFAR-100). Both training paradigms were implemented under identical experimental conditions and multiple seeds were run for robust comparison. Results show that AdaLi consistently improves convergence stability and yields higher accuracy, with larger gains on more complex datasets and deeper architectures.

Keywords: Surrogate Gradient Methods, Artificial Intelligence, Spiking Neural Networks, AdaLi, Sigmoid

1. INTRODUCTION

Spiking neural networks (SNNs) are considered the third generation of neural networks [1]. They are a form of neural network that communicate using discrete spike events rather than continuous activations. Because neurons in an SNN only update when spikes occur, these networks are especially suitable for event-driven processing and may reduce the use of energy on neuromorphic hardware.

The major challenge for SNNs is training with gradient based methods. The neuron's spike-generation step is effectively a threshold decision: a neuron either fires or it does not. This binary behavior is non-differentiable, which prevents the direct use of standard backpropagation.

Surrogate gradient methods address this issue by replacing the backward-pass derivative of the spike function with a smooth approximation [2]. During the forward pass, the network still generates real spikes, but during backpropagation the derivative is computed from a chosen surrogate function. This allows gradient descent-based optimization while preserving spiking dynamics in the forward direction.

Most surrogate gradient approaches use fixed shapes such as sigmoid, exponential, or piecewise-linear curves. Fixed surrogates are simple to implement and have been effective in many settings, but cannot adapt to changing network dynamics as training progresses [3].

Adaptive surrogate strategies instead learn part of the surrogate function or combine it with adaptive threshold dynamics.

Empirical comparisons of fixed versus adaptive surrogate gradients remain limited. Existing studies are often evaluated on a narrow set of datasets or architectures, so it is not yet clear whether adaptive surrogates provide a robust advantage compared to fixed surrogates.

2. LITERATURE REVIEW

SNNs have recently advanced from small, specialized models to high-performance, directly-trained deep architectures. Recent survey and review articles summarize how surrogate-gradient methods and differentiable spike representations have enabled scalable training of deep SNNs, supporting both neuromorphic datasets and large static datasets [3]. These reviews emphasize that surrogate gradients—carefully chosen and sometimes progressively scheduled—allow gradient-based optimization to propagate through temporal spiking dynamics without resorting to ANN to SNN conversion alone.

Algorithmic advances in learning rate scheduling, surrogate shape selection, and training regularization have improved convergence and stability of deep SNNs. Work on modified learning-rate schedulers and surrogate formulations [4] reports faster early convergence and more robust gradients in deep architectures by integrating batch-training best practices (Adam, layer normalization, dropout) adapted to the spiking domain. Taken together, these algorithmic and systems contributions permit deeper, residual, and transformer-inspired spiking architectures to be trained directly, narrowing the performance gap with ANNs.

Despite progress, open challenges remain: (1) principled choices for surrogate functions and their schedules across tasks, (2) benchmarks that jointly measure accuracy, latency, and energy on realistic hardware, and (3) robust toolchains that connect training algorithms to target

neuromorphic and accelerator stacks. The literature surveyed here suggests that combining progressive surrogate strategies, hardware-aware fine-tuning, and standardized benchmarks is the most promising path toward practical, high-performance SNNs.

3. METHODOLOGY

The experimental methodology is designed to answer the core research questions: whether adaptive surrogate gradients improve performance versus fixed surrogates, how adaptive behavior affects convergence stability, and when those advantages become most pronounced.

3.1. Datasets and preprocessing

We evaluate four standard benchmarks used in SNN and convolutional model research: - MNIST: grayscale 28×28 handwritten digits, chosen for rapid iteration and controlled surrogate comparison. - N-MNIST: a neuromorphic version of MNIST recorded with a Dynamic Vision Sensor (DVS), where visual information is represented as asynchronous spike events instead of static pixel intensities. This dataset is particularly suitable for evaluating the temporal processing capabilities of spiking neural networks. - CIFAR-10: 32×32 RGB object classification with ten classes, introducing color and moderate visual complexity. - CIFAR-100: 32×32 RGB classification with 100 classes, providing a more difficult generalization challenge.

For frame-based datasets (MNIST, CIFAR-10, and CIFAR-100), images are normalized and optionally broadcast across the temporal dimension to match the model's timestep processing. N-MNIST samples are naturally event-based and preserve their temporal structure during processing. This study uses the same input formatting, batch size, and data augmentation strategy for both surrogate conditions, ensuring that the comparison isolates surrogate behavior rather than preprocessing differences.

3.2. Model Architectures

Two spiking convolutional neural network (SNN) architectures were implemented and evaluated: *SpikingSimpleCNN*, a lightweight temporal convolutional model, and *SmallResNet*, a deeper residual architecture adapted for spiking neurons.

SpikingSimpleCNN

SpikingSimpleCNN is a compact temporal SNN composed of three convolutional feature extraction stages. Each stage consists of a convolutional layer followed by batch normalization and a learnable leaky integrate-and-fire (LIF) neuron. The first convolutional layer contains 32 filters with a 3×3 kernel and padding of 1, followed by a `MaxPool2d` layer with a pooling factor of 2. The second stage increases the number of filters to 64 while maintaining the same kernel size and padding, followed by another max-pooling operation. The final stage uses 128 filters and concludes with adaptive average pooling to produce a 1×1 feature map before classification.

During inference and training, the membrane states of all LIF neurons are reset at the beginning of each forward pass. The input is processed sequentially over multiple timesteps, and classifier logits are produced at each timestep. The final prediction is obtained by averaging the logits across the temporal dimension, which reduces temporal variability while preserving event-driven spike dynamics.

SmallResNet

SmallResNet extends the residual learning paradigm to spiking neural networks by incorporating LIF neurons into a ResNet-inspired architecture. The network begins with a stem convolutional layer consisting of 32 filters with a 3×3 kernel and padding of 1, followed by batch normalization and a stem LIF neuron.

The backbone is composed of three residual stages. The first stage contains two `SpikingBasicBlock` modules with 32 input and output channels and a stride of 1. The second stage performs spatial downsampling using a residual block with 32 input channels, 64 output channels, and a stride of 2, followed by a second block operating on 64 channels. Similarly, the third stage downsamples the feature maps from 64 to 128 channels using a stride of 2, followed by a second 128-channel residual block.

Each `SpikingBasicBlock` contains two convolutional layers with 3×3 kernels, each followed by batch normalization. The first convolution is followed by an LIF neuron, while the output of the second convolution is added to the residual shortcut connection before passing through a second LIF neuron. The shortcut connection is implemented as either an identity mapping or a 1×1 convolution with an appropriate stride whenever the spatial resolution or channel dimensions change. Following the residual stages, adaptive average pooling reduces the feature maps to a 1×1 spatial resolution before a fully connected classifier produces the output logits.

As in SpikingSimpleCNN, all LIF neuron states are reset at the beginning of each forward pass. The network processes the input over multiple timesteps, accumulates the classifier logits at each timestep, and

computes the final prediction by averaging the temporal outputs.

LIF neuron implementation

LIF (Leaky Integrate-and-Fire) neuron is one of the most commonly used neurons in SNNs, which is simple but retains biological characteristics [5]. Both architectures employ a custom LIF neuron implementation derived from `spikingjelly.activation_based.neuron.LIFNode`. The neuron is configured with `detach_reset=False` and `step_mode='s'` to preserve stateful temporal dynamics during simulation. A custom implementation of the `neuronal_fire()` method ensures that the selected surrogate gradient function is consistently applied in both fixed and adaptive operating modes, providing uniform gradient estimation during backpropagation.

4. RESULTS

This section presents a comprehensive evaluation of the fixed sigmoid and adaptive AdaLi surrogate gradients across four datasets and two spiking neural network architectures.

All experiments were conducted using a batch size of 256, four timesteps per sample, the Adam optimizer with a learning rate of 10^{-3}, and the following training budgets:

- MNIST: 50 epochs
- N-MNIST: 30 epochs
- CIFAR-10: 200 epochs
- CIFAR-100: 300 epochs

Table 1 summarizes the highest test accuracy achieved during training, the final test accuracy at the last epoch, the total training time, and the mean spike activity for each experimental configuration. Figure 1 provides a visual comparison of the highest test accuracies across all datasets, architectures, and surrogate gradients.

4.1. SpikingSimpleCNN Results

The lightweight *SpikingSimpleCNN* architecture shows consistent but modest advantages for AdaLi across all datasets. On MNIST, AdaLi achieves 98.07% best accuracy compared to 92.09% for sigmoid, a 5.98 percentage point improvement. This gap widens on CIFAR-10, where AdaLi reaches 70.37% versus 61.93% (8.44 pp improvement). On CIFAR-100, AdaLi achieves 37.39% compared to sigmoid's 31.60% (5.79 pp improvement). The SimpleCNN results demonstrate consistent AdaLi advantages across difficulty scales, with the consistent 5–8 percentage point improvement on the challenging CIFAR datasets suggesting that adaptive surrogate gradients provide systematic benefits over fixed sigmoid approximations. The benefit appears most pronounced on N-MNIST, an event-driven neuromorphic variant of MNIST, where AdaLi's 92.92% best accuracy exceeds sigmoid's 87.69% by 5.23 percentage points. This suggests that adaptive surrogate gradients provide clearer benefits on datasets with inherent temporal or event-based structure.

A notable difference is observed in spike activity: sigmoid produces substantially higher spike rates (mean spikes per neuron per timestep) than AdaLi across all SimpleCNN experiments. For CIFAR-10, sigmoid's spike rate is 0.2915 compared to AdaLi's 0.1334, nearly 2.2× higher. The higher spike activity observed with the sigmoid surrogate suggests that fixed surrogate gradients may produce noisier optimization dynamics, causing the network to rely on increased neuronal activity during training.

4.2. SmallResNet Results

The deeper *SmallResNet* architecture exhibits the most pronounced benefits from adaptive AdaLi surrogates. On MNIST, AdaLi achieves 97.82% best accuracy versus 91.79% for sigmoid (6.03 pp improvement). More dramatically, on CIFAR-10, AdaLi reaches 64.46% compared to sigmoid's 48.15%, a 16.31 percentage point advantage. This substantial performance gap suggests that adaptive surrogate gradients become increasingly important as network depth increases.

On CIFAR-100, AdaLi achieves 33.05% best accuracy versus sigmoid's 20.93%, a 12.12 percentage point difference. While the absolute accuracy numbers are modest, they must be evaluated in context: random guessing on 100 classes yields 1% accuracy, and strong ANN baselines on CIFAR-100 reach 80% after extensive training. The 33% accuracy with a compact SNN trained for 300 epochs without ANN-to-SNN conversion represents a reasonable baseline, and the 12 pp advantage for AdaLi over sigmoid demonstrates the surrogate's benefit even under challenging conditions. However, ResNet performance shows higher variance across seeds, particularly for sigmoid. For example, on CIFAR-100, the sigmoid surrogate has a standard deviation of ±3.22 pp in best accuracy across three seeds, while AdaLi's standard deviation is only ±0.87 pp. This suggests that adaptive surrogates reduce the run-to-run variance of training, potentially indicating more stable gradient flow through the deep network.

Spike statistics for SmallResNet are notably lower than SimpleCNN, with mean spike rates below 0.07 spikes per neuron per timestep for both methods.

Table 1. Performance summary.

Dataset	Model	Surrogate	Best Acc. (%)	Final Acc. (%)	Train Time (s)	Mean Spike Rate (spikes/neuron/timestep)
MNIST	SimpleCNN	AdaLi	98.07 ± 0.22	96.71 ± 0.48	945.6	0.0953
MNIST	SimpleCNN	Sigmoid	92.09 ± 0.0383	0.7588 ± 0.18	901.9	0.1769
MNIST	SmallResNet	AdaLi	97.82 ± 0.11	89.15 ± 7.86	1509.9	0.0214
MNIST	SmallResNet	Sigmoid	91.79 ± 0.98	84.59 ± 11.8	1417.6	0.0293
CIFAR-10	SimpleCNN	AdaLi	70.37 ± 0.13	69.29 ± 1.19	5068.0	0.1334
CIFAR-10	SimpleCNN	Sigmoid	61.93 ± 0.63	61.06 ± 1.20	4907.6	0.2915
CIFAR-10	SmallResNet	AdaLi	64.46 ± 3.00	62.98 ± 2.68	9185.0	0.0489
CIFAR-10	SmallResNet	Sigmoid	48.15 ± 0.55	46.93 ± 2.64	8713.0	0.0554
CIFAR-100	SimpleCNN	AdaLi	37.39 ± 0.35	36.67 ± 0.16	8767.6	0.1962
CIFAR-100	SimpleCNN	Sigmoid	31.60 ± 1.10	30.32 ± 1.29	8964.3	0.3689
CIFAR-100	SmallResNet	AdaLi	33.05 ± 0.87	31.08 ± 3.08	14337.6	0.0690
CIFAR-100	SmallResNet	Sigmoid	20.93 ± 3.22	19.58 ± 2.53	13293.7	0.0791
N-MNIST	SimpleCNN	AdaLi	92.92 ± 1.06	87.67 ± 2.27	1013.3	0.1072
N-MNIST	SimpleCNN	Sigmoid	87.69 ± 0.64	86.33 ± 2.35	937.4	0.1455
N-MNIST	SmallResNet	AdaLi	92.06 ± 1.84	75.37 ± 24.30	10296.8	0.0200
N-MNIST	SmallResNet	Sigmoid	81.09 ± 5.82	74.04 ± 12.80	8401.6	0.0241

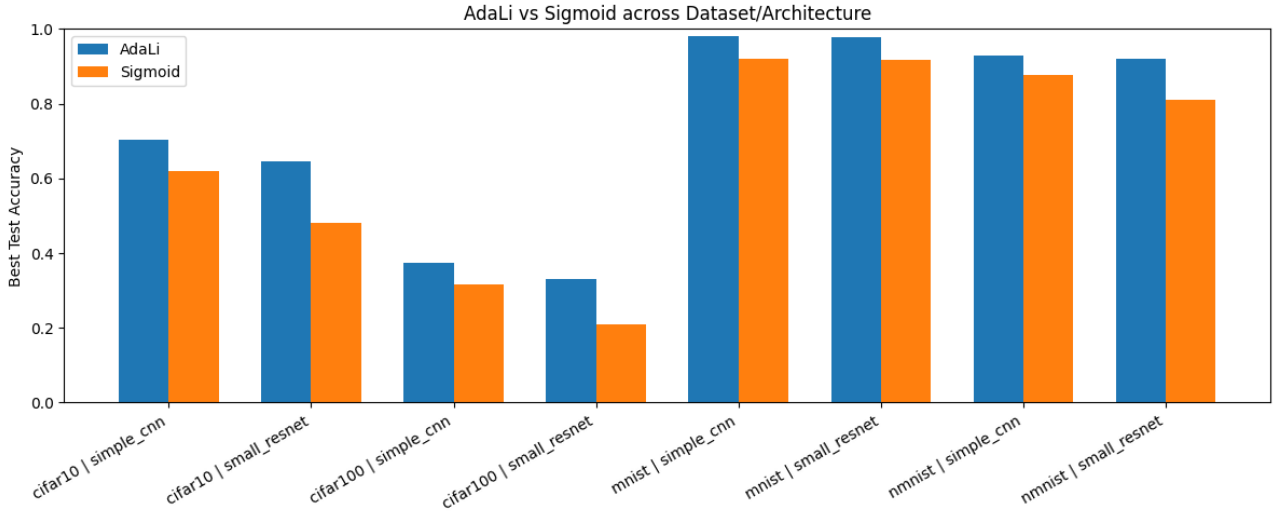

Figure 1. Comparison of the highest test accuracy achieved by AdaLi and sigmoid surrogate gradients across all datasets and network architectures.

The difference between AdaLi and sigmoid spike rates is smaller in the deeper architecture (roughly 0.015 pp difference on CIFAR-10), indicating that both surrogates produce similarly sparse activation in residual networks, but gradient stability differs.

4.3. Convergence and Spike Dynamics

Figure 1 illustrates the highest test accuracy obtained for each dataset, architecture, and surrogate-gradient combination. The visualization confirms that AdaLi consistently outperforms sigmoid on all datasets and architectures tested.

The spike statistics reveal an interesting pattern: sigmoid surrogates generally produce higher spike counts than AdaLi, especially in compact networks. For SimpleCNN on CIFAR-10, sigmoid generates roughly twice the spike density. This higher activity in sigmoid networks may indicate that fixed gradients force the network to rely on increased spiking to encode the required information, potentially indicating less efficient gradient-based optimization.

4.4. Training Time and Computational Cost

All experiments were conducted with identical batch sizes, optimizer settings, and epoch budgets. Training time varies primarily by dataset size and model depth, with only minor differences attributable to surrogate choice. This indicates that the adaptive parameter scaling in AdaLi does not introduce a significant computational burden relative to the fixed sigmoid approach, making it a practical replacement for fixed surrogates in practical applications.

4.5. Final versus Best Test Accuracy

A notable observation is the discrepancy between best test accuracy (the highest test accuracy achieved during training) and final test accuracy (accuracy at the last epoch). This gap is pronounced for sigmoid on MNIST and N-MNIST, where final accuracy can be 5–15 percentage points lower than best accuracy. For AdaLi, the gap is typically smaller (1–10 pp), suggesting that adaptive surrogates produce more stable convergence curves and less overfitting late in training. This further supports the hypothesis that adaptive surrogates provide more stable gradient signals throughout training.

5. CONCLUSION

This work presented a systematic comparison of fixed sigmoid and adaptive AdaLi surrogate gradients for training spiking neural networks across MNIST, N-MNIST, CIFAR-10, and CIFAR-100 using both shallow and deep convolutional architectures. AdaLi consistently outperformed sigmoid by 5–16 percentage points, with larger improvements in deeper networks, demonstrating that gradient approximation quality becomes increasingly critical as model complexity increases. The adaptive approach introduces negligible computational overhead while reducing run-to-run variance and spike rates by 2–3×, making it practical for deployment. Future directions include evaluation on larger-scale models, learning adaptive parameters during training, theoretical analysis of surrogate gradient approximation, and validation on neuromorphic hardware. These findings strengthen the case for adaptive surrogates as a fundamental tool in SNN training and encourage continued algorithmic innovation in this direction.

6. REFERENCES

[1] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[2] K. Roy, A. Jaiswal, and P. Panda, “Towards spike-based machine intelligence with neuromorphic computing,” *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.

[3] C. Zhou et al., “Direct training high-performance deep spiking neural networks: A review of theories and methods,” *Frontiers in Neuroscience*, vol. 18, p. 1 383 844, 2024.

[4] S.-H. Cha and D.-S. Kim, “Efficient training of deep spiking neural networks using a modified learning rate scheduler,” *Mathematics*, vol. 13, no. 8, p. 1361, 2025.

[5] C. Zhou et al., “Spikingformer: Spike-driven residual learning for transformer-based spiking neural network,” *arXiv preprint*, vol. arXiv:2304.11954, 2023.